Backend Developer

홍 승 현

유도의 수 읽기로 설계하는

데이터 무결성 특화 백엔드 엔지니어

Law-Partner · MedNeduro · AI 에이전트 보안 연구

github.com/HONG-SH-1 · ghdtmdgus2245@naver.com

CORE PHILOSOPHY

"동작 원리를 모르는 코드는 시한폭탄이다"

— 홍승현의 개발 원칙 · AI 코드도 블랙박스 없이 내부 메커니즘을 완벽히 검토하여 적용하는 것을 지향

04:30

새벽 4시 30분

비전공자의 한계를 매일의
몰입으로 극복해 온
압도적 실행력

⚡

AI 블랙박스 해체

AI가 제안한 코드도
내부 메커니즘 완벽 검토 후
적용하는 원칙

🥋

유도의 수 읽기

상대 빈틈을 먼저 읽는
분석력을 방어적 아키텍처
설계에 이식

CORE COMPETENCIES

핵심 무기 5가지

01 **Spring Security / JWT**

RTR + HttpOnly 쿠키 + XSS·CSRF 동시 방어

02 **AES-256 무중단 암호화**

JPA Converter 자동화 + Key Rotation (서비스 중단 없는 키 교체)

03 **Redis Rate Limiter**

글로벌 분산 트래픽 통제 · JMeter 스트레스 검증 (JMeter 1만 TPS 검증 완료)

04 **AOP 감사 로그 & Zero-trust**

@ActionLog 커스텀 어노테이션 + FK 단절 무결성 + 3단계 폴백 설계

05 **DB 설계 / Apache Tika 파일 보안**

ERD 정규화 · 해시 컬럼 분리 · 바이너리 MIME 검증 원천 차단

PROJECT OVERVIEW

Law-Partner

법률 플랫폼 — 민감 데이터 보호 특화 백엔드

개발 기간	2026.01 – 2026.03 (약 2개월)
팀 구성	5인 팀 / 보안 리드 · DB 설계 · 관리자 페이지 담당
도메인	법률 상담 플랫폼 (일반사용자 / 변호사 / 관리자 3-tier)
핵심 과제	민감 개인정보 보호 · 관리자 행위 감사 · 비정상 트래픽 차단

1차 프로젝트 — MedNeduro (의료 플랫폼)

Spring Interceptor · IP 마스킹 · 전체 DB 설계 · 3인 팀 / 보안 리드 github.com/HONG-SH-1/MedNeduro

TECH STACK

- Spring Boot
- Spring Security
- JWT / RBAC
- Oracle DB
- JPA / MyBatis
- FastAPI
- React
- WebSocket

SYSTEM ARCHITECTURE

시스템 아키텍처 — 보안 레이어 중심

보안 필터 · 인증/인가 · 암호화 · DB 설계

Client

Browser

HTTP Request with JWT

Security Gate (HSH)

- IP Blacklist Filter
- Rate Limiter (Bucket4j)
- JWT Auth Filter
- RBAC Check

Spring Boot Core (HSH)

- Admin Controller (Zero-trust)
- AOP Audit Logger (@Async)
- AES-256 Converter (JPA Auto)
- Masking AOP (Response)

AI Server

FastAPI
LangChain
LLM

Oracle DB (HSH)

Oracle
AES-256
암호화 저장
FK 단절 설계

Redis (Global Rate Limiter) 예정

ERD & 데이터 설계 철학

01 의도적 FK 단절 — 로그 무결성 영구 보장

관리자 계정이 삭제되어도 감사 로그(TB_ADMIN_AUDIT)가 함께 삭제되지 않도록 USER 테이블과의 외래키를 의도적으로 맺지 않음. 금융·법률 도메인의 컴플라이언스 요구사항 직접 반영.

02 검색용 SHA-256 해시 컬럼 분리

이메일·전화번호를 AES-256으로 암호화하여 저장하되, 중복 체크·검색을 위한 SHA-256 해시 컬럼을 별도 관리. 암호화(보안)와 검색 성능을 동시에 확보하는 구조 설계.

03 3NF 정규화 & 인덱스 설계

3NF 정규화로 데이터 중복 제거 및 정합성 확보. 자주 조회되는 컬럼(이메일 해시, 상태코드, 역할코드)에 인덱스 적용하여 조회 성능 최적화.

[문제] JWT 취약점 & DB 키 유출 대참사 위험

문제 1. XSS 취약점 직접 발견

JWT를 LocalStorage에 저장하던 기존 방식.

악성 JS 삽입(XSS) 시 토큰 탈취 → 계정 완전 장악 가능.

개발 중 브라우저 콘솔에서 토큰을 직접 확인하며 보안 구조 자체에 결함이 있음을 스스로 인지.

위험 시나리오

XSS 삽입 → JS에서 localStorage.getItem('token') → 토큰 외부 전송 → 무기한 계정 도용

문제 2. 암호화 키 유출 대참사

개발 중 실수로 암호화 키 값을 분실하는 사건 발생.

이 순간 인식: 실제 운영 환경에서 키가 유출되면 DB의 모든 민감 데이터가 한 번에 노출됨.

단순 암호화로는 부족하다.
키 생명주기 관리 전략이 반드시 필요하다.

필요한 해결책

구버전 키로 암호화된 데이터를 신버전 키로 무중단 전환하는 Key Rotation 전략 설계 필요

→ 이 두 문제가 8장(JWT RTR)과 9장(AES-256 Key Rotation)의 해결책으로 이어집니다

[해결 1] JWT RTR + XSS/CSRF 동시 방어 구조

BEFORE

LocalStorage

Access Token ⚠️
Refresh Token ⚠️

❌ XSS 공격 시 전체 탈취
❌ CSRF 취약

→

AFTER — RTR (Refresh Token Rotation)

```
AuthService.java — RTR 핵심 로직
// HttpOnly 쿠키에서 Refresh Token 추출
String oldRT = null;

for (Cookie c : request.getCookies()) {

  if ("refreshToken".equals(c.getName()))
    oldRT = c.getValue();
}


// DB 토큰과 대조 (토큰 탈취 감지)
if (!savedToken.getTokenValue()
    .equals(hash(oldRT))) {

  refreshTokenService.deleteToken(savedToken);

  throw new CustomException(INVALID_TOKEN);
}
```

```
// 즉시 무효화 — Race Condition 방어
refreshTokenService.deleteByUserNo(userNo);


// 새 토큰 발급 + DB 저장 (RTR 완성)
TokenDTO newToken = jwtTokenProvider
    .createToken(auth, userNo, ...);

refreshTokenService.save(userNo, newToken);
```

[해결 2] AES-256 JPA Converter + 무중단 키 로테이션

Aes256Converter.java

```java
@Component
@Converter  // JPA 자동 연동

public class Aes256Converter
    implements AttributeConverter<String,String>{

  // Entity → DB: 자동 암호화
  @Override
  public String convertToDatabaseColumn
      (String attribute) {

    return aes256Util.encrypt(attribute);
  }

  // DB → Entity: 자동 복호화
  @Override
  public String convertToEntityAttribute
      (String dbData) {

    // 탈퇴 유저 방어 — 서비스 무중단
    if (dbData.startsWith("delete_"))
      return dbData;

    return aes256Util.decrypt(dbData);
  }
}
```

KeyRotationService.java

```java
@Profile("migration") // 운영 완전 분리

public class KeyRotationService {

  // 구버전·신버전 키 동시 주입
  @Qualifier("oldAesUtil") Aes256Util old;
  @Qualifier("newAesUtil") Aes256Util next;


  public void rotateKeys() {
    for (User user : userRepository.findAll()) {
      try {
        // 구 키 실패 시 신 키로 폴백
        String plain = decryptWithOldFirst
            (user.getEmail());

        user.setEmail(next.encrypt(plain));

      } catch (Exception e) {

        // 1명 실패해도 전체 계속!
        log.error("유저 처리 오류", e);
      }
    }
  }
}
```

CASE 02 · ISSUE

[문제] 서버 확장(Scale-out) 시 보안 정책 우회 및 서비스 마비 위험

01

분산 환경 정합성 붕괴

WAS 서버가 2대로 스케일 아웃될 경우, 각 서버의 인메모리 카운터가 독립 동작
→ 동일 IP가 WAS-1에 50회, WAS-2에 50회씩 총 100회 공격 가능. Bucket4j 인메모리 방식의 구조적 한계.

02

동기식 슬랙 알림의 스레드 병목

비정상 IP 탐지 시 슬랙 API 호출이 동기적으로 처리됨
→ 슬랙 응답 대기(1~3초) 동안 서버 스레드 점유 → 대량 공격 시 스레드 풀 고갈로 서비스 전체 마비 위험.

03

해결 방향 — Redis + @Async 이벤트 분리

Redis를 글로벌 Rate Limiter 저장소로 도입
→ 모든 WAS가 하나의 카운터 공유. ApplicationEventPublisher로 알림 로직을 완전 분리 → 서버 스레드 즉시 반환.

[해결] Redis 글로벌 Rate Limiter + 비동기 알림 파이프라인

Redis Rate Limiter (설계)

```
// WAS 1·2가 동일 Redis 카운터 공유
// → 분산 환경 정합성 완벽 보장

RateLimiter limiter = RateLimiter
  .create(redisTemplate, ipKey);

if (!limiter.tryConsume(1)) {

  // 50회 초과 → 즉시 429 응답
  response.setStatus(429);


  // 이벤트 발행 → 스레드 즉시 반환!
  eventPublisher.publishEvent(
    new BlockedIpEvent(clientIp));


  return;
}
```

@Async 비동기 이벤트 처리 (설계)

```
// 알림 로직을 완전히 별도 스레드로!
@EventListener
@Async  // 서버 스레드와 완전 분리
public void handleBlockedIp(
    BlockedIpEvent event) {

  String ip = event.getIp();

  // DB 블랙리스트 등록
  blacklistRepo.save(new BlacklistIp(ip));

  // 슬랙 알림 (지연돼도 서버 무관)
  slackClient.sendAlert(
    "차단 IP: " + ip);
}
```

JMeter 스트레스 테스트 — 설계

테스트 시나리오: 동일 IP에서 1초에 10,000건 로그인 요청 발송

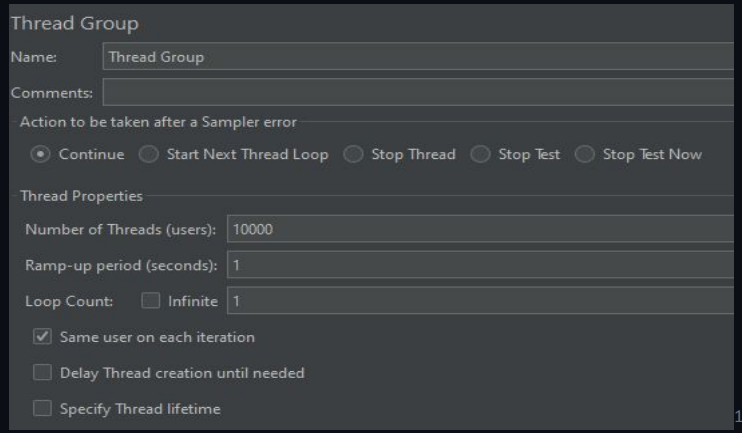
1분 내 300회 이상 요청 시 자동 차단 로직이 정확히 동작하는지 검증

Thread 수	Ramp-up	Loop	Target API
10,000	1초	1회	POST /auth/login

예상 결과

300건만 200 OK
나머지 전부 429 Too Many Requests 차단

측정 환경



⚙️ 검증 완료

Rate Limiter 핵심 로직 및 차단 정밀도 검증 (Single WAS)

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename | Browse... | Log/Display Only: Errors Successes | Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput ↑	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Requ...	10000	50	0	2209	267.42	98.35%	2683.1/sec	7158.26	14.58	2731.9
TOTAL	10000	50	0	2209	267.42	98.35%	2683.1/sec	7158.26	14.58	2731.9

Text

HTTP Request

Sampler result | Request | Response data

Response Body | Response headers

Find | Case sensitive

{"success": false, "code": "SEC-429", "message": "요청이 너무 자주 발생했습니다. 잠시 후 다시 시도해주세요."}

1만 TPS 100% 중 98.35% 방어

CODE : SEC-259

MESSAGE : 요청이 너무 자주 발생했습니다...

Text

HTTP Request

Sampler result | Request | Response data

Thread Name:Thread Group 1-352
Sample Start:2026-03-30 13:15:49 KST
Load time:102
Connect Time:1
Latency:102
Size in bytes:14316
Sent bytes:127
Headers size in bytes:421
Body size in bytes:13895
Sample Count:1
Error Count:0
Data type ("text"|"bin"|""):text
Response code:200
Response message:

HTTPSampleResult fields:
ContentType: application/json
DataEncoding: null

⚙️ 테스트 결과

1. 테스트 시나리오

단일 IP에서 1초 내 1,000건의 무차별 대입(Brute-force) 성격의 로그인/조회 요청 발송

2. 검증 결과

차단 성공률 : 98.35% (비정상 트래픽의 압도적 대다수를 필터 레이어에서 즉각 격리)

응답 정황성 : 차단된 모든 요청에 대해 정의된 SEC-429 보안 응답 코드 반환 확인

3. 엔지니어링 인사이트

Fail-Closed 정책 구현 : 시스템 가용성 한계를 초과하는 트래픽 유입 시, 인메모리 [Bucket4j] 기반 필터가 최우선 적으로 작동하여 내부 서버 자원(CPU/DB) 고갈을 원천 차단

향후 도전 과제

1. 분산 환경 정합성 확보 (Redis 전환)

- **내용**: 현재 단일 WAS 인메모리 방식(Bucket4j)의 구조적 한계를 극복하기 위해 Redis 기반 글로벌 카운터 도입.

- **목표**: 서버가 스케일 아웃(Scale-out)되어도 모든 인스턴스가 동일한 300회/분 임계치를 공유하도록 설계하여 보안 사각지대 제거.

2. 초고부하 상황에서의 차단 정밀도 최적화

- **내용**: 10,000 TPS 이상의 극한 부하 유입 시 발생하는 네트워크 병목 및 필터 오버헤드 분석.

- **목표**: 시스템 가용 자원에 따라 임계치를 유연하게 조절하는 동적(Dynamic) 레이트 리밋 알고리즘을 연구하여, 서비스 중단 없는 방어 체계 구축.

CASE 03 · ISSUE

[문제] 관리자 권한 남용 추적 불가 + 악성 파일 우회

관리자 행위 추적 불가 — 컴플라이언스 위반

- 관리자의 회원 정지·승인·삭제 등 민감 작업 수행 시 누가 언제 무엇을 했는지 기록 전혀 없음.
- 감사 로그 부재 → 내부 권한 남용, 실수에 의한 데이터 변경을 사후 추적 불가.
- 금융·법률 도메인에서는 이런 구조 자체가 컴플라이언스 위반.

확장자 위조 악성 파일 업로드 공격

- 단순 확장자 검사(.pdf, .jpg)는 악성 스크립트를 malware.pdf로 이름만 바꿔 우회 가능.
- 변호사 자격증 파일 업로드 기능 존재 → 파일 업로드 공격 벡터 직접 노출.
- 서버에서 파일이 실행되거나 DB에 악성 경로 저장 시 시스템 전체 위협.

[해결 1] @ActionLog AOP 감사 로그 & FK 단절 무결성 설계

LogingAspect.java

```java
// 모든 컨트롤러 자동 감시
@Around("execution(* ..*Controller.*(..))")
public Object logAccess(PJP jp) throws Throwable{

  // 요청 고유 식별자 (추적용)
  String traceId = UUID.randomUUID()
    .toString().substring(0, 8);

  MDC.put("TRACE_ID", traceId);

  } finally { // 성공·실패 무조건 기록!

    adminAuditRepository.save(
      AdminAudit.builder()

        .adminNo(adminNo)   // 관리자 식별
        .actionType(actionType)
        .traceId(traceId)  // 요청 추적
        .errorYn(errorYn) // 실패도 기록
        .reason(reason)   // 처리 사유
        .build());
  }
}
```

FK 단절 — 로그 무결성 영구 보장

TB_USER ──✗── TB_ADMIN_AUDIT

관리자 계정이 삭제되어도
감사 로그는 영구 보존됨.
의도적 FK 미설정으로 구현.

관리자 식별 3단계 폴백

1단계: JWT details → userNo (DB 조회 X)
2단계: CustomUserDetails → getUserNo()
3단계: 최후 — DB에서 이메일로 직접 조회

→ 어떤 환경에서도 감사 기록 누락 방지

```
2026-03-30T18:59:40.477+09:00  INFO 1 --- [o-8080-exec-522] c.e.backend_main.HSH.aop.LogingAspect    : [Audit] TraceID: dfbcd212, URI: /a
pi/admin/logs/threats, Status: 200, Time: 5ms
2026-03-30T19:00:10.375+09:00  INFO 1 --- [o-8080-exec-516] c.e.backend_main.HSH.aop.LogingAspect    : [Audit] TraceID: 98ceca9c, URI: /a
pi/admin/users, Status: 200, Time: 65ms
2026-03-30T19:00:10.438+09:00  INFO 1 --- [o-8080-exec-573] c.e.backend_main.HSH.aop.LogingAspect    : [Audit] TraceID: 96a6c288, URI: /a
pi/admin/summary, Status: 200, Time: 32ms
2026-03-30T19:00:10.468+09:00  INFO 1 --- [o-8080-exec-430] c.e.backend_main.HSH.aop.LogingAspect    : [Audit] TraceID: 103209ff, URI: /a
pi/admin/logs/threats, Status: 200, Time: 4ms
2026-03-30T19:00:40.393+09:00  INFO 1 --- [o-8080-exec-532] c.e.backend_main.HSH.aop.LogingAspect    : [Audit] TraceID: 686512cf, URI: /a
pi/admin/users, Status: 200, Time: 67ms
2026-03-30T19:00:40.461+09:00  INFO 1 --- [o-8080-exec-522] c.e.backend_main.HSH.aop.LogingAspect    : [Audit] TraceID: efaf957c, URI: /a
pi/admin/summary, Status: 200, Time: 31ms
2026-03-30T19:00:40.497+09:00  INFO 1 --- [o-8080-exec-516] c.e.backend_main.HSH.aop.LogingAspect    : [Audit] TraceID: ee7b393f, URI: /a
pi/admin/logs/threats, Status: 200, Time: 5ms
```

tb_admin_audit ×

Properties | DATA | Log | ER | Monitor | { } | fourjo2

```
select * from tb_admin_audit LIMIT 100
```

Search Results | Export

* ACTION_TYPE varchar(30)	* TARGET_INFO varchar(200)	REASON varchar(200)	TRACE_ID varchar(50)

[해결 2] Apache Tika 바이너리 검증 + IP 블랙리스트 캐시 설계

LawyerService.java — Tika 검증

```java
// 확장자가 아닌 바이너리 자체를 분석!
private final Tika tika = new Tika();

String mimeType = tika.detect(
    file.getInputStream());

// malware.pdf → 실제 타입: text/html
if (!mimeType.startsWith("image/")
    && !mimeType.equals("application/pdf")) {

  log.warn("⛔ 차단: {}", mimeType);
  throw new IllegalArgumentException(
    "이미지/PDF만 허용합니다.");
}


// UUID로 파일명 예측 불가 처리
String savedName = UUID.randomUUID()
    + "_" + originalName;
```

IpBlacklistFilter.java

```java
// 스레드 세이프 메모리 캐시
private final Set<String> cache =
    ConcurrentHashMap.newKeySet();

private static final long TTL = 60_000; // 1분

// DB가 아닌 메모리에서 즉시 차단! (I/O 제로)
if (cache.contains(ip)) {

  response.setStatus(SC_FORBIDDEN);

  response.getWriter().write(
    "{code: BL-403}");
  return;
}

// 더블 체킹 락 — 동시성 완벽 방어
synchronized (this) {

  // TTL 만료 시 DB 재동기화
  if (expired) refreshFromDB();
}
```

구현 결과 — 운영 투명성 & 보안 자동화 확보

100%
감사 로그
기록률

0건
미인증 관리자
접근 통과

BL-403
차단 IP
즉시 응답

UUID
파일명
예측 불가

1단계

- 데이터 소스
JWT Claims

- 목적 및 보안 가치
DB조회 없이
토큰에서 userNo
즉시 추출

2단계

- 데이터 소스
Security Context

- 목적 및 보안 가치
현재 인증된 세션
정보를 통한 관리자
특정.

3단계

- 데이터 소스
최후의 보루(DB)

- 목적 및 보안 가치
정보 유실 시 이메일
기반 최종 조회로
기록 누락 방지.

"보안은 기능 구현보다 예외 상황에서의 연속성 확보가 본질이다."

단순히 로그를 남기는 기능을 넘어,
시스템 장애나 비정상 요청 환경에서도 반드시 증거를 남기는 **방어적 감사 아키텍처**를 완성

사일로(Silo) 현상 — 팀원 이탈 리스크 선제 방어

Problem

특정 팀원에게 기획·핵심 로직이 독점되는 사일로 현상 포착

1차 프로젝트(MedNeduro) 초기, 전체 기획과 핵심 로직을 특정 팀원이 독점 담당
→ 해당 팀원 없이는 전체 개발 방향을 아무도 모르는 구조임을 간파.
"팀원 중 누가 빠지더라도 시스템은 흔들림 없이 굴러가야 한다"는 원칙을 수립.

Action

교차 검증(Cross-check) 문화 직접 제안 및 주도

정기 미팅에서 서로의 코드·기획을 교차 리뷰하는 컨텍스트 동기화 문화 도입.
단순 승인이 아닌 상대방 로직을 직접 설명할 수 있는 수준까지 이해를 요구.

Result

프로젝트 중반 팀원 예고 없이 이탈 → 남은 3명이 빈자리 즉각 흡수, 일정 지연 없이 프로젝트 성공적으로 완료

교차 검증 프로세스 — 컨텍스트 동기화 구조

STEP 1

전체 기획 방향 합의

매 스프린트 시작 전 전체 시스템 흐름을 팀원 전원이 함께 검토.
담당자만 아는 기획이 없도록 모든 의사결정을 공유 문서로 기록. 누구도 블랙박스가 되지 않는 구조.

STEP 2

상대방 코드 리뷰

자신이 작성하지 않은 코드를 반드시 한 명 이상이 리뷰. 단
순 승인이 아닌 로직 흐름을 직접 설명할 수 있는 수준까지 이해를 요구. 이해하지 못하면 리뷰 통과 불가.

STEP 3

이탈 시 즉각 흡수 → 성공적 완료

교차 검증 덕분에 팀원 이탈 발생 시 남은 팀원들이 빈자리를 즉각 파악하고 업무 분담 완료.
일정 지연 없이 프로젝트를 정상 마무리. '누가 빠져도 시스템은 돌아간다'는 원칙 실현.

"견고한 팀은 개인의 능력이 아닌, 시스템이 누군가의 이탈을 흡수할 수 있는가로 결정된다."

Git 롤백 대참사 — 휴먼 에러를 시스템으로 방어

Problem 완성된 핵심 보안 로직이 Git에서 통째로 롤백되는 사고 반복 발생

2차 프로젝트(Law-Partner) 중간 발표 직전, Git에 익숙하지 않은 팀원의 실수로 병합 완료된 JWT 인증 로직·AES 암호화 코드가 원격 저장소에서 통째로 롤백되는 사고 발생. 동일 사고 2회 반복.

시각 전환: '팀원의 실수'가 아닌 '시스템의 부재'로 진단

책임 추궁 대신, 누구라도 실수할 수 있는 환경이 문제임을 인식. '이 실수가 구조적으로 불가능한 환경'을 만드는 것으로 방향 전환.

1차 방어: 패키지 물리적 격리: 팀원 각자의 이니셜(HSH, LDJ 등)로 작업 패키지와 폴더를 완전 분리
→ 서로 다른 폴더를 건드리지 않는 구조로 병합 충돌 원천 차단.

2차 방어: 페어 프로그래밍으로 근본 원인 동기화: 직접 팀원 옆에 앉아 어떤 명령어에서 코드가 꼬였는지 병합 과정을 함께 역추적. 덮어씌우는 대신 원인을 이해하고 안전하게 올리는 법을 동기화.

패키지 격리 구조 + 코드 유실율 0% 달성

IntelliJ 패키지 구조 (이니셜 분리)

```
backend_main/

├── common/          # 공통 모듈 (보안/엔티티)
│   ├── security/    # JWT, Filter
│   ├── entity/      # JPA Entities
│   ├── converter/   # AES Converter
│   └── aop/         # LogingAspect
│
├── HSH/             ◂ 홍승현 전담 영역
│   ├── controller/
│   ├── service/     # AuthService.java
│   └── migration/   # KeyRotation
│
├── LDJ/             ◂ 팀원 전담 영역
│   └── ...
└── [기타팀원]/
    └── ...
```

"견고한 시스템은 뛰어난 코드보다 방어적 프로세스에서 완성된다."

0% 코드 유실률 (격리 도입 후)

0회 병합 충돌 (구조적 차단)

4회→0회 반복 사고 → 구조로 해결

RETROSPECTIVE

한계점 & 다음 스텝

01 **Redis Rate Limiter — 분산 환경 완성**

현재: 단일 WAS 인메모리 Bucket4j (스케일 아웃 시 정합성 문제 존재)

→ **Redis 기반 글로벌 카운터 구현 + JMeter 10,000건 스트레스 검증**

02 **Kafka 도입 — 대규모 비동기 파이프라인**

현재: @Async + ApplicationEventPublisher 수준의 비동기 알림 처리

→ **Kafka 메시지 큐로 감사 로그·알림 파이프라인을 엔터프라이즈 수준으로 확장**

03 **AI 에이전트 보안 거버넌스 — 국제 논문**

현재: Spring AOP/Proxy 기반 보안 게이트웨이 설계 및 Latency 최적화 연구 진행 중

→ **International Conference 논문 등재 (2026 하반기 목표)**

데이터 무결성과 보안을 코드에 녹여내는 백엔드 엔지니어
함께 단단한 시스템을 만들어갈 팀을 기다리고 있습니다.

감사합니다

홍승현 | ghdtmdgus2245@naver.com | github.com/HONG-SH-1 | https://hong-sh-1.github.io

JWT RTR | AES-256 | Redis (WIP) | IP Blacklist | Apache Tika | AOP Audit